

Guía de Prácticas – Semana 4 (2h)

El mini-proyecto integra los cuatro subtemas de la clase 4:

- ES6+: `let/const`, arrow functions, template literals, destructuring, módulos.
- Variables/funciones/estructuras de control.
- Asincronía con Promesas y `async/await`.
- Manejo de errores con `try/catch/finally`, errores personalizados, logs y fallbacks.

La práctica crea una página que:

1. Valida un formulario simple (nombre + email).
2. Simula peticiones a una API (con **Fetch** y **fallback local**).
3. Muestra estados de **carga/éxito/error** y registra errores.

1) Estructura de directorios

```
semana4-practica/  
├─ index.html  
├─ styles.css  
├─ js/  
│   ├── app.js           # Punto de entrada (type="module")  
│   ├── api.js           # Fetch API + Promesas + async/await + manejo de errores  
│   ├── validation.js    # Validaciones (email, required) + errores personalizados  
│   ├── utils.js         # Helpers: spinner, logs, render, sleep()  
│   └─ mock/  
│       └─ users.json    # Datos de respaldo (fallback offline)  
└─ assets/  
    └─ logo.svg          # Opcional (placeholder)
```

Sugerencia de ejecución: abrir con Live Server (VS Code) o doble clic en `index.html` (para que `fetch` del `users.json` funcione localmente, Live Server es lo ideal).

2) Código de todos los archivos

`index.html`

```
<!doctype html>  
<html lang="es">  
<head>  
  <!-- Codificación estándar -->  
  <meta charset="utf-8" />  
  <!-- Viewport responsable del escalado en móviles -->  
  <meta name="viewport" content="width=device-width, initial-scale=1" />  
  <title>Semana 4 · Práctica ES6+, Promesas y Errores</title>  
  <!-- Hoja de estilos principal -->  
  <link rel="stylesheet" href="./styles.css" />  
  <!-- Respetar modo claro/oscuro del sistema -->  
  <meta name="color-scheme" content="light dark">  
</head>  
<body>  
  <!-- Encabezado del sitio -->  
  <header class="site-header" role="banner">  
    <div class="container">  
      <h1>Práctica - Semana 4</h1>  
      <p class="subtitle">ES6+ · Promesas · async/await · Manejo de errores</p>  
    </div>  
  </header>  
  
  <!-- Contenido principal -->  
  <main class="container" role="main">  
    <!-- Bloque A: Formulario con validación en cliente -->
```

```

<section class="card" aria-labelledby="form-title">
  <h2 id="form-title">A. Validación de formulario</h2>
  <p>Ingresa tus datos y valida en cliente (sin enviar al servidor).</p>

  <!-- novalidate desactiva validación HTML nativa para controlar mensajes -->
  <form id="user-form" novalidate>
    <!-- Campo: nombre -->
    <div class="field">
      <label for="name">Nombre <span aria-hidden="true">*</span></label>
      <input id="name" name="name" type="text" placeholder="Tu nombre" required />
      <!-- Contenedor para error accesible del campo nombre -->
      <small class="error" data-error-for="name" aria-live="polite"></small>
    </div>

    <!-- Campo: email -->
    <div class="field">
      <label for="email">Email <span aria-hidden="true">*</span></label>
      <input id="email" name="email" type="email" placeholder="tucorreo@ejemplo.com"
required />
      <!-- Contenedor para error accesible del campo email -->
      <small class="error" data-error-for="email" aria-live="polite"></small>
    </div>

    <!-- Acciones del formulario -->
    <div class="actions">
      <button type="submit" class="btn">Validar</button>
      <button type="reset" class="btn btn-secondary">Limpiar</button>
    </div>

    <!-- Mensaje global del formulario (éxito/fracaso) -->
    <div class="feedback" id="form-feedback" aria-live="polite"></div>
  </form>
</section>

<!-- Bloque B: Carga de datos (API vs mock local) -->
<section class="card" aria-labelledby="api-title">
  <h2 id="api-title">B. Carga de usuarios (Fetch + async/await)</h2>
  <p>Puedes alternar entre una API pública y un <em>mock</em> local.</p>

  <div class="stack">
    <!-- Toggle que cambia la fuente de datos -->
    <label class="toggle">
      <input type="checkbox" id="use-mock" />
      <span>Usar <strong>mock local</strong> (./js/mock/users.json)</span>
    </label>

    <!-- Botones de acción para cargar o forzar error -->
    <div class="actions">
      <button id="btn-load" class="btn">Cargar usuarios</button>
      <button id="btn-load-error" class="btn btn-danger">Probar error</button>
    </div>

    <!-- Estado textual (cargando/ok/error) -->
    <div id="status" class="status" aria-live="polite"></div>
    <!-- Listado de usuarios renderizados -->
    <ul id="users" class="users" aria-live="polite"></ul>
  </div>
</section>

<!-- Bloque C: Consola/registro simple en UI -->
<section class="card" aria-labelledby="log-title">
  <h2 id="log-title">C. Registro de eventos y errores</h2>
  <pre id="logs" class="logs" aria-live="polite"></pre>
</section>
</main>

<!-- Pie de página -->
<footer class="site-footer" role="contentinfo">
  <div class="container">
    <small>© 2025 - Semana 4 · ES6+, Promesas y Errores</small>
  </div>
</footer>

<!-- Punto de entrada JS. type="module" habilita import/export -->
<script type="module" src="./js/app.js"></script>
</body>
</html>

```

styles.css

```
/* Box-model coherente en toda la app */
* { box-sizing: border-box; }

/* Tokens de diseño: espacios, color y énfasis */
:root{
  --space-1:.5rem; --space-2:1rem; --space-3:1.5rem; --space-4:2rem;
  --border:#e5e7eb; --text:#111; --muted:#6b7280;
  --brand:#2563eb; --ok:#16a34a; --warn:#ca8a04; --err:#dc2626;
}

/* Tamaño base de fuente (accesible) */
html { font-size: 100%; }

/* Tipografía y colores base */
body {
  margin: 0;
  font-family: system-ui, -apple-system, Segoe UI, Roboto, sans-serif;
  line-height: 1.5;
  color: var(--text);
}

/* Contenedor centralizado y con padding */
.container { max-width: 72rem; margin-inline:auto; padding: var(--space-2); }

/* Separadores superior/inferior del sitio */
.site-header, .site-footer { border-block:1px solid var(--border); }
.subtitle { color: var(--muted); margin-top: .25rem; }

/* Tarjetas visuales para agrupar secciones */
.card {
  border:1px solid var(--border);
  border-radius:.75rem;
  padding: var(--space-3);
  margin-block: var(--space-3);
  background: #fff;
}

/* Campos de formulario */
.field { margin-bottom: var(--space-2); }
.field label { display:block; font-weight:600; margin-bottom:.25rem; }
.field input {
  width:100%; padding:.6rem .75rem;
  border:1px solid var(--border); border-radius:.5rem;
}

/* Línea reservada para el mensaje de error por campo */
.error { color: var(--err); min-height: 1lh; display:block; margin-top:.25rem; }

/* Grupo de botones responsivo */
.actions { display:flex; gap:.5rem; flex-wrap:wrap; margin-top: .5rem; }
.btn { background: var(--brand); color:#fff; border:0; padding:.6rem .9rem; border-radius:.5rem; cursor:pointer; }
.btn-secondary { background: #6b7280; }
.btn-danger { background: var(--err); }

/* Layout vertical simple para el bloque API */
.stack { display:grid; gap: var(--space-2); }
.toggle { display:flex; align-items:center; gap:.5rem; }

/* Estados coloreados para feedback textual */
.status { min-height:1lh; color: var(--muted); }
.status.loading { color: var(--warn); }
.status.success { color: var(--ok); }
.status.error { color: var(--err); }

/* Lista de usuarios renderizados */
.users { list-style:none; padding:0; margin:0; display:grid; gap:.5rem; }
.users li { border:1px solid var(--border); border-radius:.5rem; padding:.6rem .75rem; }

/* Consola de logs simple */
.logs {
  background:#0b1020; color:#c7d2fe; padding: var(--space-2);
  border-radius:.5rem; min-height: 8rem; overflow:auto;
}
```

```

/* Tema oscuro: respeta preferencia del sistema */
@media (prefers-color-scheme: dark) {
  body { color:#e5e7eb; background:#0f172a; }
  .card { background:#0b1224; border-color:#1f2a44; }
  .site-header, .site-footer { border-color:#1f2a44; }
  .field input { background:#0b1224; color:#e5e7eb; border-color:#1f2a44; }
  .users li { border-color:#1f2a44; }
}

```

js/utils.js

```

// ===== Utilidades ES6+ compartidas =====

// Selección rápida de nodos (querySelector)
export const $ = (sel, root = document) => root.querySelector(sel);
// Selección múltiple convertida a array
export const $$ = (sel, root = document) => [...root.querySelectorAll(sel)];

// Sleep: promesa que resuelve tras 'ms' (útil en mocks/demos)
export const sleep = (ms = 700) => new Promise(res => setTimeout(res, ms));

// Pinta estado textual en la UI (cargando/ok/error)
export const renderStatus = (msg = "", type = "") => {
  const el = $("#status");
  if (!el) return;
  el.textContent = msg;
  el.className = `status ${type}`.trim();
};

// Renderiza la lista de usuarios
export const renderUsers = (users = []) => {
  const list = $("#users");
  list.innerHTML = users
    .map(u => `- <strong>${u.name}</strong> · <small>${u.email}</small></li>`)
    .join("");
};

// Limpia listado
export const clearUsers = () => { const list = $("#users"); if (list) list.innerHTML = ""; };

// Log minimalista con timestamp e indicador de tipo
export const log = (message, kind = "info") => {
  const area = $("#logs");
  const stamp = new Date().toISOString();
  const tag = kind.toUpperCase().padEnd(5);
  area.textContent += `[${stamp}] ${tag} ${message}\n`;
  area.scrollTop = area.scrollHeight; // auto-scroll
};

// Error de aplicación con metadatos (para enriquecer manejo de errores)
export class AppError extends Error {
  constructor(message, meta = {}) {
    super(message);
    this.name = "AppError";
    this.meta = meta; // ej: {status, url, code, cause}
  }
}

```

js/validation.js

```

// ===== Validaciones de formulario y error personalizado =====
import { AppError } from "../utils.js";

// Chequea vacío (string vacío, null, undefined)
export const isEmpty = v => !v || !String(v).trim();

// Validación básica de email (regex razonable para práctica)
export const isEmail = v => /^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(String(v).trim());

// Valida un objeto con los campos del formulario.
// Si hay errores, lanza AppError con detalle por campo.

```

```

export function validateForm({ name, email }) {
  const errors = {};

  if (isEmpty(name)) errors.name = "El nombre es obligatorio";
  if (isEmpty(email)) errors.email = "El email es obligatorio";
  else if (!isEmail(email)) errors.email = "Formato de email inválido";

  if (Object.keys(errors).length) {
    // Cortamos el flujo y entregamos detalle para UI
    throw new AppError("Validación fallida", { errors });
  }
  return { ok: true };
}

```

js/api.js

```

// ===== Capa de datos: Fetch + async/await + manejo de errores =====
import { AppError, log, sleep } from "../utils.js";

// URL remota (demo pública)
const REMOTE_URL = "https://jsonplaceholder.typicode.com/users";
// Fallback local para offline/demo controlada
const MOCK_URL = "../js/mock/users.json";

// Carga usuarios desde API o mock.
// forceError=true simula una falla controlada para probar manejo de errores.
export async function loadUsers({ useMock = false, forceError = false } = {}) {
  if (forceError) {
    await sleep(500); // simula tiempo de red
    throw new AppError("Fallo simulado en la carga", { code: "FORCED_ERROR" });
  }

  const url = useMock ? MOCK_URL : REMOTE_URL;

  try {
    // Petición HTTP (acepta JSON)
    const res = await fetch(url, { headers: { "Accept": "application/json" } });

    // Si el servidor responde con error (4xx/5xx), lanzamos AppError
    if (!res.ok) {
      throw new AppError(`HTTP ${res.status}`, { status: res.status, url });
    }

    // Intento de parseo JSON (también podría fallar si el body no es JSON)
    const data = await res.json();

    // Normalización de estructura (garantiza name/email)
    const users = data.map(u => ({
      id: u.id,
      name: u.name ?? `${u.firstName ?? "N"} ${u.lastName ?? ""}.trim()`,
      email: u.email ?? "no-email@example.com",
    }));

    return users;
  } catch (err) {
    // Si no es AppError, lo envolvemos para unificar manejo arriba
    if (!(err instanceof AppError)) {
      throw new AppError(err.message || "Error desconocido al cargar usuarios", { cause: err });
    }
    // Si ya era AppError, lo propagamos tal cual
    throw err;
  }
}

```

js/app.js

```

// ===== Punto de entrada de la app (módulo) =====
import { $, log, renderStatus, renderUsers, clearUsers } from "../utils.js";
import { validateForm } from "../validation.js";
import { loadUsers } from "../api.js";

// Cache de nodos de UI

```

```

const form = $("#user-form");
const formFeedback = $("#form-feedback");
const btnLoad = $("#btn-load");
const btnLoadError = $("#btn-load-error");
const useMockToggle = $("#use-mock");

// --- Helpers de UI para mostrar errores por campo ---
function setFieldError(name, message = "") {
  // El <small> destinatario del mensaje:
  const el = $('[data-error-for="' + name + '"]');
  if (el) el.textContent = message;
  // Atributo aria-invalid para accesibilidad
  const input = $('#${name}');
  if (input) input.setAttribute("aria-invalid", message ? "true" : "false");
}

function clearFormErrors() {
  ["name", "email"].forEach(n => setFieldError(n, ""));
  formFeedback.textContent = "";
  formFeedback.className = "feedback";
}

function setFormFeedback(msg, type = "ok") {
  formFeedback.textContent = msg;
  formFeedback.className = `feedback ${type}`;
}

// --- A. Validación del formulario (cliente, sin envío real) ---
form.addEventListener("submit", e => {
  e.preventDefault(); // Evita recarga/navegación
  clearFormErrors();

  // Recogemos datos de inputs
  const data = {
    name: $("#name").value,
    email: $("#email").value
  };

  try {
    // Valida y lanza AppError si encuentra errores
    validateForm(data);

    // Si no lanzó, todo OK
    setFormFeedback("✅ Datos válidos. (Simulación: listos para enviar)", "ok");
    log(`Formulario validado: ${JSON.stringify(data)}`, "info");
  } catch (err) {
    // Si trae metadatos de validación, pintamos por campo
    if (err?.meta?.errors) {
      const { errors } = err.meta;
      if (errors.name) setFieldError("name", errors.name);
      if (errors.email) setFieldError("email", errors.email);
      setFormFeedback("❌ Corrige los campos indicados.", "error");
      log(`Validación fallida: ${JSON.stringify(errors)}`, "error");
    } else {
      // Error inesperado (no de validación controlada)
      setFormFeedback("❌ Error inesperado al validar", "error");
      log(`Error inesperado: ${err.message}`, "error");
    }
  }
});

// --- B. Carga de usuarios con async/await (API o mock) ---
async function handleLoad({ forceError = false } = {}) {
  // Limpia UI previa
  clearUsers();
  renderStatus("Cargando usuarios...", "loading");
  log(`Inicio carga usuarios (mock=${useMockToggle.checked}, error=${forceError})`, "info");

  try {
    // Carga y espera respuesta
    const users = await loadUsers({
      useMock: useMockToggle.checked,
      forceError
    });
  }
}

```

```

    // Si todo va bien, renderizamos
    renderUsers(users);
    renderStatus(`Se cargaron ${users.length} usuarios.`, "success");
    log(`Usuarios cargados: ${users.length}`, "info");
  } catch (err) {
    // Cualquier error controlado/inesperado llega aquí
    renderStatus(`Error: ${err.message}`, "error");
    log(`ERROR carga usuarios: ${err.message} | meta=${JSON.stringify(err.meta ?? {})}`,
"error");
  } finally {
    // Siempre se ejecuta
    log("Operación finalizada (finally)", "info");
  }
}

// Enlazamos botones a handler
btnLoad.addEventListener("click", () => handleLoad());
btnLoadError.addEventListener("click", () => handleLoad({ forceError: true }));

```

js/mock/users.json

```

[
  { "id": 1, "name": "Ada Lovelace", "email": "ada@ejemplo.com" },
  { "id": 2, "name": "Alan Turing", "email": "alan@ejemplo.com" },
  { "id": 3, "name": "Grace Hopper", "email": "grace@ejemplo.com" }
]

```

(Opcional) js/mock/users.jsonc — ¡Solo para lectura comentada!

```

[
  // Datos de ejemplo para fallback local / modo offline
  { "id": 1, "name": "Ada Lovelace", "email": "ada@ejemplo.com" },
  { "id": 2, "name": "Alan Turing", "email": "alan@ejemplo.com" },
  { "id": 3, "name": "Grace Hopper", "email": "grace@ejemplo.com" }
]

```